

COMP426 - Assignment 1

Patrick Deniso (40194944)

I implemented Conway's Game of Life (optionally multi-species) in C++20. The CPU does all the simulation; OpenGL is used only to display pixels at ~30 FPS. The program has two roles:

- **Control thread (main):** owns timing and rendering.
- **Computation threads (workers):** update the grid in parallel.

Data Layout

- Grid size: 1024×768, row-major indexing $i = y \cdot W + x$.
- Double buffering per species: `layers_curr[s]` (read-only this frame) and `layers_next[s]` (write-only).
- This avoids locks: all threads read the same “old” state and write to separate memory.

Computation Thread (Worker) Design

- Spawn `T` workers (`std::thread`; default = `hardware_concurrency()`).
- Partition by **row blocks**: each thread gets a disjoint `[y0, y1)` range → no write overlap, good cache locality.
- For each frame and each species:
 1. Count 8 neighbors from `layers_curr[s]`.
 2. Apply Life rule (S23/B3).
 3. Store into `layers_next[s]`.

- Synchronize with a `std::barrier<>` (participants = $T + 1$). Each worker calls `arrive_and_wait()` once per frame.

Control Thread (Main) Design

Per frame:

1. `barrier.arrive_and_wait()` to ensure workers finished writes.
2. **Swap** buffers once (`next <-> curr`) to make the new world current.
3. Build an RGBA image from `curr` (species→color mapping).
4. Upload with `glTexSubImage2D` and draw a full-screen quad (OpenGL/GLFW/GLAD).
5. Pace to ~30 FPS with `sleep_until(t0 + 33ms)`.

Why barrier without completion?

Using `std::barrier<>` (no completion functor) is portable on macOS libc++ and keeps the single swap in an obvious place (right after the sync on main).

Optimizations

- **Lock-free hot path:** read-only `curr`, disjoint writes to `next`.
- **Cache-friendly iteration:** `y` outer, `x` inner.
- **Compiler flags:** `-O3 -march=native -DNDEBUG`.
- **Thread count knob:** set to physical cores.
- Optional: toroidal wrapping for livelier dynamics; higher density for longer transients.

Correctness & Behavior

- Double buffering guarantees correct neighbor reads.

- Fixed seed for deterministic runs (given same thread count and boundary mode).
- On a finite non-wrapping grid, random worlds typically settle into still lifes and small oscillators (expected “ash”).

Build & Run (macOS, Homebrew)

```
cmake -S . -B build -Dglfw3_DIR=/opt/homebrew/opt/glfw/lib/cmake/glfw3
cmake --build build -j
./build/game [species] [workers]
```

Complexity

Per frame: $O(S \times W \times H)$ cell updates; scales well across CPU cores due to no contention.